

Table of Contents

Regulatory Radar — IBM Bobathon Challenge

Build an AI agent that watches EU regulations so humans don't have to: it reads a stream of regulatory updates, works out **which companies and products each change affects**, and **alerts them** with a clear, actionable message. Built with **IBM Bob**; alerts fired with **Twilio**.

GenAI Builders Day · GDGoC TUM Campus Heilbronn · partner challenge by EcoComply

💬 **Questions any time?** Join the event WhatsApp group:
<https://chat.whatsapp.com/BQf8Eul1t2gA7LCaBD1z2Q>

⚠️ Hosts/jury, read first

This repo contains a `jury/` folder with the **answer key**. **Do not push jury/ to a public repository**. A `.gitignore` already excludes it, but double-check before you publish the link. Share the repo *without* `jury/`; keep `jury/` in a private place for scoring.

TL;DR

EU product regulations (RoHS, REACH, the Battery Regulation, PPWR, RED, GPSR, ...) change constantly. Miss one and a company faces fines, blocked shipments, or delisting. Today, a lot of that monitoring is done by hand. **Your job: automate it**. Turn a messy feed of regulatory updates into the right alert, to the right company, about the right product, before the deadline.

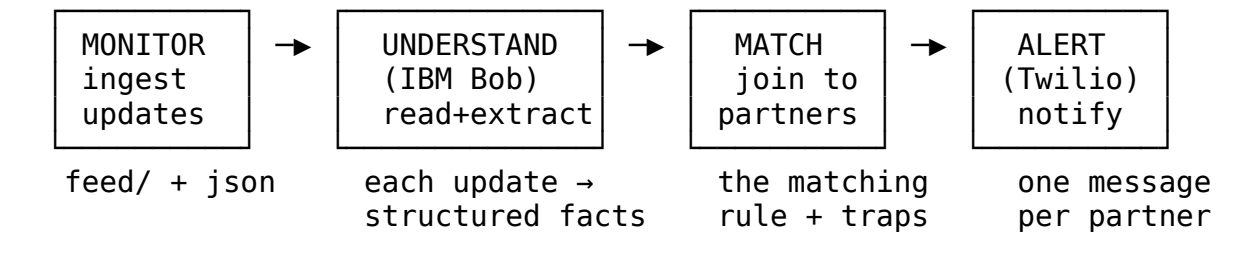
You get a realistic synthetic dataset (companies + products + a regulatory feed). A working end-to-end demo on a slice of it beats a big plan.

Why this matters (the story)

EcoComply is a Heilbronn startup that helps electronics SMEs stay EU-market-ready. One of their services is *continuous monitoring of regulatory updates* — and a lot of it is still manual: people read legislation portals, map each change to clients by hand, and email them one at a time. When a rule changes, the affected company needs to know **today**, not when a listing gets pulled. That mapping-and-alerting loop is exactly what you're going to automate.

What you'll build

A four-stage agent pipeline:



Quickstart

```
git clone <this-repo>
cd regulatory-radar
python3 starter.py           # loads the data, runs a naive baseline
                             # matcher, self-checks one update
```

Then: 1. Get your **IBM Bob** access (30-day free): <https://ibm.biz/student-bobathon> 2. Grab your **Twilio** credit — promo code **TUM-TWILIO-50** (for the alert step). 3. Open this repo in Bob and ask it to read `README.md` and `DATASET_README.md`, then help you build the pipeline (see [Working with IBM Bob](#)).

The dataset

Everything you need is in this repo. Full field-by-field schema is in **`DATASET_README.md`**.

File	What it is
<code>partners.json / partners.csv</code>	22 synthetic electronics SMEs and their 53 products — the entities to match against
<code>regulatory_updates.json</code>	50 regulatory update events — the feed to process
<code>taxonomy.json</code>	Controlled vocabulary (product categories, substances, regulation families)
<code>feed/</code>	10 simulated EUR-Lex/ECHA-style HTML pages + an index, for practising the scrape/“Monitor” step
<code>sample_expected_output.json</code>	The expected matcher output for one update, so you know the shape to produce
<code>starter.py</code>	A runnable scaffold: loads the data, a naive baseline matcher, and a self-check
<code>dataset_stats.json</code>	Summary counts

By the numbers: 22 partners · 53 products · 50 updates (44 substantive, 6 pure noise, 6 duplicates/corrections) · deadlines from already-overdue to 2028.

Everything — companies, products, dates, emails, phone numbers — is **fabricated and safe**. Contacts use `@example.com` and placeholder phone numbers, so test alerts can never reach a real person. Use your **own** Twilio test number/email for the Alert demo.

The challenge, step by step

This is what we expect you to deliver. Each step maps to part of the [scoring rubric](#).

Step 1 — Monitor (ingest the feed)

Read the regulatory updates. The easy path is to load `regulatory_updates.json` directly. For extra realism, scrape the HTML pages in `feed/` instead — that simulates pulling from a live regulatory portal. *(A saved sample like this is a perfectly acceptable stand-in for a live feed.)*

Output of this step: a list of update objects your pipeline can work on.

Step 2 — Understand (use IBM Bob to read each update)

For each update, use **IBM Bob** to read the free-text summary and pull out the facts you need: - regulation family, the deadline (`deadline_date`), severity; - which **product categories**, **substances**, and **markets** it affects; - any **conditions** (e.g. “only LMT/industrial batteries”, “only products sold in Germany”); - whether it’s a **duplicate/correction** of an earlier update (`corrects` field).

The structured `scope` block is given to help you, but the real signal is in the human-written summary and conditions. Using Bob to interpret them is the point of this step.

Output of this step: each update reduced to clean, structured criteria.

Step 3 — Match (join updates to affected products)

This is the heart of the challenge (and the biggest scoring weight). A product is **affected** when **all** of these hold:

1. **Market overlap** — the product’s markets include a market the update covers (EU = all 27 states).
2. **Category in scope** — the update targets the product’s category (or targets “all”).
3. **Substance present** — if the update names substances, the product actually contains one.
4. **Attribute conditions** — e.g. battery type, has-radio, connector, intended use, packaging.
5. **Not excluded** — e.g. GPSR excludes medical/industrial-only equipment.

The feed is deliberately messy, like a real one. To score well you must handle: - **Irrelevant updates** from other domains → match no one. - **In-scope updates that still affect nobody** in this portfolio → return an empty match, don’t guess. - **Duplicates/corrections** → alert once, not twice (use the `corrects` field). - **Precision traps** → products that look related but are just out of scope: wrong market, a substance that isn’t actually present, or an attribute (battery type, intended use) that excludes them. Matching on keywords alone will over-fire here.

Output of this step: for each update, the list of affected `{partner_id, product_id, reason, deadline}` — see `sample_expected_output.json` for the exact shape.

Step 4 — Alert (fire a real notification)

For each affected partner, send **one** clear, actionable message on their `preferred_channel` (email / SMS / WhatsApp): “*New rule X affects your product Y. Deadline Z. Here’s what to do.*” **Twilio** is the quick way to fire a real SMS/WhatsApp/email — promo code **TUM-TWILIO-50**.

Replace the placeholder contacts with **your own** Twilio test number/verified email. At least one **real** alert firing in your demo is the wow moment.

Output of this step: a real notification (and ideally a log of what was sent to whom and why).

Difficulty tiers (pick your level)

You don’t have to do everything. Aim for a tier and nail it.

Tier	Who	Do this
Beginner	New to coding/AI	Steps 1 → 3 → 4 using the structured scope fields only: match category + substance + market, then fire one alert for a couple of updates. Skip the free-text and the traps.
Core (target)	Most teams	Use Bob to read the free-text summary (Step 2), handle noise and duplicates , respect markets and attribute conditions , and fire real alerts. This is the full loop.
Stretch	Strong teams	Add any of: a portfolio risk dashboard , deduplication across the whole feed, deadline-based prioritisation , multi-language alerts, an audit log , or false-positive control on the precision traps.

A correct **Core** solution beats a flashy-but-wrong Stretch one.

Working with IBM Bob

Bob is both the thing you build with *and* a reasoning engine inside your pipeline. A good first move: open this repo in Bob and ask it to read `README.md` + `DATASET_README.md` and scaffold the pipeline.

Example prompts (copy, paste, adapt):

- **Orient:** “*Read README.md and DATASET_README.md in this repo and summarise, in your own words, the four steps I need to build and the data files involved.*”

- **Understand step:** “Here is a regulatory update: <paste one object from regulatory_updates.json>. Extract a JSON with: family, deadline, affected_categories, affected_substances, affected_markets, conditions, and is_duplicate (from the corrects field).”
 - **Match step:** “Given this product <paste a product> and this update scope <paste scope>, does the update apply? Walk through market overlap, category, substance, and attribute conditions, and give a yes/no with the deciding condition.”
 - **Build the matcher:** “Write a Python function that loads partners.json and regulatory_updates.json and returns, per update, the affected products using: market overlap (EU = all 27 states), category in scope, substance present if listed, and attribute conditions. Skip updates flagged as duplicates via corrects.”
 - **Alert step:** “Draft a concise SMS (under 300 characters) telling partner X that update Y affects product Z, including the deadline and one recommended action.”
-

Tools

- **IBM Bob — required.** IBM’s AI coding agent. Build the pipeline fast *and* use it as the reasoning engine for the Understand/Match steps. IBM mentors are on-site all day. (30-day free access: <https://ibm.biz/student-bobathon>)
- **Twilio — recommended for alerts.** Fastest way to fire a real SMS/WhatsApp/email. Promo code **TUM-TWILIO-50**. Not mandatory — any channel that delivers a real alert counts.
- **Anything else** — any language, scraping lib, or framework you like. Keep the scope to a thin end-to-end slice.

What to submit

- **A working prototype** — a live demo where an update flows through detect → match → and a **real alert fires**.
- **Your matches output** — a JSON (same shape as sample_expected_output.json) covering the updates you processed.
- **A short README** — what it does, your stack, and exactly how you used **IBM Bob** and **Twilio**.
- **A 3-min demo + 1-min pitch** to the jury.

How you’re judged

Criterion	What we look for	Weight
Works end-to-end	Detect → match → alert actually fires live	30%
Smart matching	Right rule → right partner, few false positives (you handled the traps)	25%
Use of IBM Bob	How effectively Bob built and powered it	15%

Criterion	What we look for	Weight
Alert delivery	A real notification fires; sensible channel	10%
Real-world fit	Would EcoComply actually use this? (actionable, auditable)	10%
Demo & communication	Clear story, crisp demo	10%

The jury scores **matching** against a hidden answer key. What moves the needle: catching the **precision traps** (right category, wrong market/substance/attribute), correctly ignoring **noise** and **zero-match** updates, and **not alerting twice** on duplicates.

For the jury / hosts (the full process)

Everything you need to score is in **jury/** (kept out of the public repo): -
jury/ground_truth.json — the full answer key: for every update, the affected products *and* the engineered near-misses, each with a reason and deadline. - **jury/JUDGING_NOTES.md** — the trap map and a step-by-step scoring guide mapped to the rubric. -
jury/regulation_match_rules.json — the exact machine rule used to derive the key, for auditing any disputed match.

Scoring flow: take each team's matches output → compare to `ground_truth.json` → check (a) precision/recall on matches, (b) correct handling of noise + zero-match updates, (c) de-duplication, (d) deadline/scope extraction, (e) a real alert firing. `JUDGING_NOTES.md` lists exactly which updates are noise, duplicates, and which carry the headline precision traps.

? FAQ

- **Do we need real regulatory data?** No. Use the provided feed (and the mock portfolio for matching). No confidential data required.
- **Do alerts have to be real?** Aim for at least one real notification firing in the demo — that's the wow moment. Twilio makes it trivial, but any channel counts.
- **Teams?** Solo or teams, formed at the event. All skill levels welcome — pick a difficulty tier.
- **What if scraping is hard?** Skip it — load `regulatory_updates.json` directly. The HTML feed is optional practice.
- **Where's the expected output format?** `sample_expected_output.json`.
- **Stuck or have a question?** Ask any mentor on the day, or post in the WhatsApp group: <https://chat.whatsapp.com/BQf8Eul1t2gA7LCaBD1z2Q>

Repo structure

```
regulatory-radar/
├── README.md           ← you are here (the whole challenge)
├── DATASET_README.md  ← full data dictionary / schemas
├── starter.py          ← runnable scaffold + naive baseline
├── partners.json / .csv ← 22 SMEs, 53 products
└── regulatory_updates.json ← 50 update events (the feed)
```

└─ taxonomy.json	← controlled vocabulary
└─ sample_expected_output.json	← expected output shape (one update)
└─ feed/	← simulated HTML regulatory pages
└─ dataset_stats.json	

(The matching answer key is held privately by the jury and is not in this repo.)

GDGoC TUM Campus Heilbronn · GenAI Builders Day · partner challenge by EcoComply · build with IBM Bob + Twilio. Now go build the radar. 🚀

Appendix — Full Data Dictionary

Synthetic dataset for the **IBM Bobathon “Regulatory Radar”** challenge (see [../challenge_brief.md](#)). Build an AI agent that **Monitors** regulatory updates, **Understands** each change, **Matches** it to the affected partners/products, and **Alerts** them.

Real EU regulation *families* are used for realism. **Every company, product, update event, date, email and phone number is fabricated and safe** - there are no real people and no live inboxes. Contacts use @example.com and placeholder phone numbers on purpose.

What’s in the box

File	Audience	What it is
partners.json	participants	22 synthetic SMEs and their 53 products (the entities to match against)
partners.csv	participants	The same portfolio, one row per product, for spreadsheets
regulatory_updates.json	participants	50 regulatory update events (the feed to process)
taxonomy.json	participants	Controlled vocabulary: categories, substances, regulation families
feed/	participants	10 simulated HTML notices for practising the scrape/“Monitor” step
sample_expected_output.json	participants	The expected matcher output for ONE update, so you know the shape to produce
jury/ground_truth.json	jury only	The full answer key: every update -> affected products + engineered near-misses
jury/regulation_match_rules.json	jury only	The exact machine rule used to derive the key

File	Audience	What it is
n		
dataset_stats.json	everyone	Summary counts
build_dataset.py	maintainers	Regenerates everything deterministically

The data model

Partner / product (`partners.json`): each partner has `sells_in` markets and a `contact` (name, email, phone, preferred_channel). Each product carries the attributes matching is based on: `category`, `substances`, `has_battery + battery_type` (portable/button_cell/lmt/industrial) + `battery_capacity_wh`, `has_radio`, `connector`, `packaging`, `intended_use` (consumer/toy/industrial/medical), and `markets`.

Regulatory update (`regulatory_updates.json`): `regulation_family`, `title`, a free-text summary (what your “Understand” step must read), `effective_date/deadline_date`, `severity`, `change_type` (new/amendment/correction), an optional `corrects` pointer (for de-duplication), and a `scope` block (affected categories / substances / markets / conditions).

How matching is defined (the rule behind the key)

A product is **affected** by an update when **all** hold: 1. **Market overlap** - the product’s markets intersect the update’s markets (EU = all 27 states); 2. **Category in scope** - the update targets the product’s category (or targets “all”); 3. **Substance present** - if the update names substances, the product contains at least one; 4. **Attribute conditions** - e.g. battery type, radio, connector, intended use, packaging; 5. **Not excluded** - e.g. GPSR excludes medical/industrial-only equipment.

A **near-miss** matches the category and market but fails *exactly one* of substance / attribute / market - these are the precision traps that punish naive keyword matching.

What to expect (and what we test)

The feed mirrors a real regulatory stream, so it is deliberately messy. Your agent should cope with:

- **Irrelevant updates** from other domains (chemicals, metals, agriculture, refrigerants). These should match no one - do not force a match.
- **Updates that affect nobody in this portfolio**, even within the electronics domain (e.g. a substance none of the partners use). The correct answer is an empty match, not a guess.
- **Duplicates and corrections** - some updates restate or correct an earlier one (see the `corrects` field). Alert once, not twice.
- **Precision traps** - products that look related but fall just outside scope: wrong market, a substance that is not actually present, or an attribute (battery type, intended use) that takes them out of scope. Matching on keywords alone will over-fire here.

Markets matter: an update scoped to a single country only affects partners that sell there. Deadlines range from already-overdue to several years out, so prioritise by `deadline_date` and `severity`.

Suggested pipeline (maps to the rubric)

1. **Monitor** - ingest `regulatory_updates.json` (or scrape `feed/*.html` for the live-feed feel).
2. **Understand** - use IBM Bob to read each `summary` and extract: family, deadline, affected categories/substances/markets, and conditions.
3. **Match** - join against `partners.json` using the rule above; mind the precision traps.
4. **Alert** - for each affected partner, send one actionable message (“Rule X affects product Y, deadline Z, do W”) on their `preferred_channel`. **Swap in your own Twilio test number/email** - do not rely on the placeholder contacts.
5. **Score (jury)** - compare against `jury/ground_truth.json`: precision/recall on matches, correct handling of noise, zero-match and duplicate updates, and deadline extraction.

Safety note on contacts

All emails (`@example.com`) and phone numbers are placeholders so that test alerts never reach a real person. Teams should configure their **own** Twilio sandbox / verified test number for the Alert demo.

Generated by `build_dataset.py` - edit the seed data there and re-run to regenerate everything.